

NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING

PLACEMENT EXAMINATION
FOR CS1010 (AND EQUIVALENT MODULES)

17 July 2019

Time Allowed: 2 hours

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains **9** questions and comprises **16** printed pages.
2. Answer **ALL** questions.
3. This is a **CLOSED BOOK** assessment. The maximum mark is 100.
4. Calculators are allowed, but not electronic dictionaries, laptops, tablets, or other computing devices.
5. Please write your **Student Number** below. Do not write your name.

Student number:

--	--	--	--	--	--	--	--	--

6. Please shade one of the circles below to indicate which module you are applying this Placement Exam for.

☐ CS1010 (in C)
☐ CS1010E (in Python)
☐ CS1010J (in Java)

☐ CS1010S (in Python)
☐ CS1101S (you may answer Section B
in C/C++, Python or Java)

This portion is for examiner's use only.

Question	Maximum	Marks
1	5	
2	5	
3	10	
4	20	
5	7	
6	8	
7	10	
8	15	
9	20	
Total	100	

Section A

1. The *Number Hive Puzzle* is a logic puzzle consisting of a block of hexagons, with different areas marked out using thicker lines. There are two rules that must hold of a completed block.
 - Each area must contain the numbers from 1 up to the number of hexagons in the area. For example, in Figure 1(a) below, the topmost area contains 4 hexagons so they must be filled with 1, 2, 3 and 4 with no repeated numbers.
 - No number can be next to the same number in any direction along a shared edge.

Figure 1(a) below shows an example and Figure 1(b) is the solved puzzle.

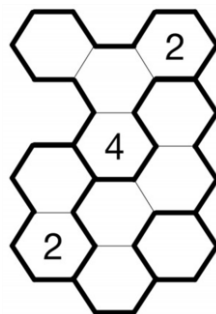


Figure 1(a)

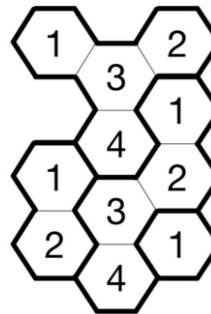


Figure 1(b)

Solve the Number Hive Puzzle in Figure 1(c) below.

[5 marks]

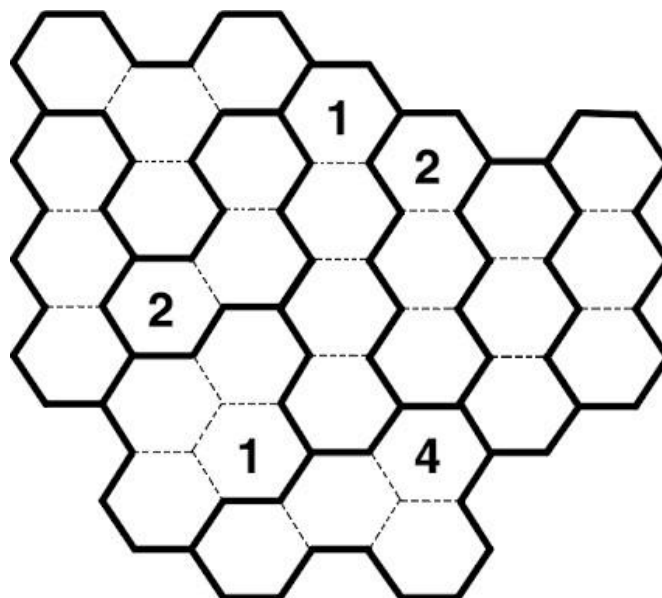


Figure 1(c)

2. [5 marks]

In a certain programming language, strings are encapsulated data objects so characters in the string cannot be manipulated directly. Hence, the language provides the following built-in functions for the manipulation of strings. The examples below assume that **str** is “computer”, **ch** is ‘A’, and **str2** is “code”.

Function	Return type	Description
length(str)	integer	Returns the length of the string str. Example: length(str) returns 8.
charAt(str, k)	character	Returns the character at index <i>k</i> in string str. The first character is at index 0. Example: charAt(str, 3) returns ‘p’.
charToStr(ch)	string	Converts the character ch into a string and returns the string. Example: charToStr(ch) returns the string “A”.
prefix(str, k)	string	Returns the first <i>k</i> characters of string str. If <i>k</i> is larger than the length of str, it returns a null string. Example: prefix(str,3) returns “com”.
suffix(str, k)	string	Returns the last <i>k</i> characters of string str. If <i>k</i> is larger than the length of str, it returns a null string. Example: suffix(str,3) returns “ter”.
concat(str, str2)	string	Catenates strings str and str2 and returns the catenated strings. Strings str and str2 are unchanged. Example: concat(str, str2) returns “computercode”.
contains(str, str2)	boolean	Returns true if string str contains string str2, or false otherwise. Example: contains(str, str2) returns false.

Using the above functions, write a pseudocode for a function **isRotate(str1, str2)** to check if **str1** is the rotation of **str2**. For example, “raining” is formed by rotating “ingrain”; “freecare” is a rotation of “carefree”; “colour” is a rotation of “colour”. The function returns true if str1 is a rotation of str2, or false otherwise. You need not use all of the functions above. The fewest functions you use, the more marks you will get. You are not allowed to use functions not listed above.

3. [10 marks]

Two strings are anagrams of each other if one can be formed by rearranging the characters of the other. For example, “design” and “signed”, “debit card” and “bad credit”, “the eyes” and “they see”. When a string is rotated, as in question 2 above, the resulting string is also an anagram.

Describe two algorithms using pseudocode to check if two strings `str1` and `str2` are anagrams. The algorithms should not be too similar.

Algorithm 1:

3. (continue...)

Algorithm 2:

4. [20 marks]

The **Blocks World** is a dynamic system consisting of blocks which could be placed on the table or stacked on top of another, forming towers. For example, given two blocks A and B, the Blocks World can be in one of the 3 states as shown in Figures 4(a), 4(b) and 4(c).

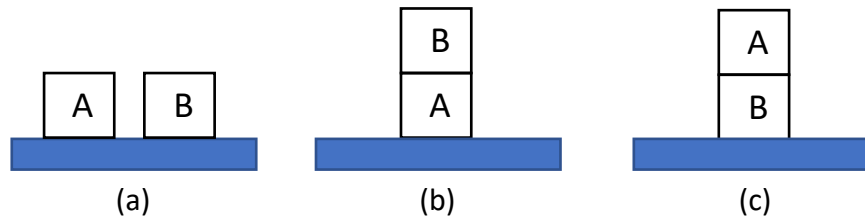


Figure 4

The actions available in the Block World are **move(x,y)** – to move block x onto y, where x is a block, and y is either the table or a block other than x. Block x must be directly above y after the move, that is, x and y must be in contact. The blocks are moved by a robot's arm, therefore only the top block in a tower can be moved. For example, in Figure 4(b), block A cannot be moved as it is not at the top of the tower. Also, in Figure 4(b), if there is another block C on the table, it cannot be moved onto block A as block C would not be in direct contact with block A.

(a) How many possible states are there given three blocks A, B and C? [2 marks]

(b) Given the initial state as shown in Figure 4(d) below, draw the final state after this sequence of actions: move(A,table), move(B,A), move(C,table), move(D,C), move(B,E), move(A,B), move(D,A). [3 marks]

Initial state:

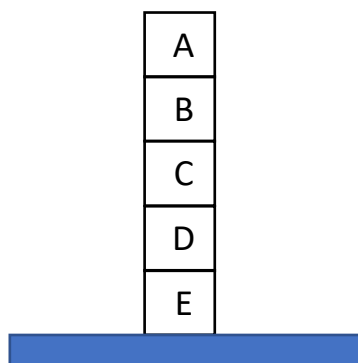
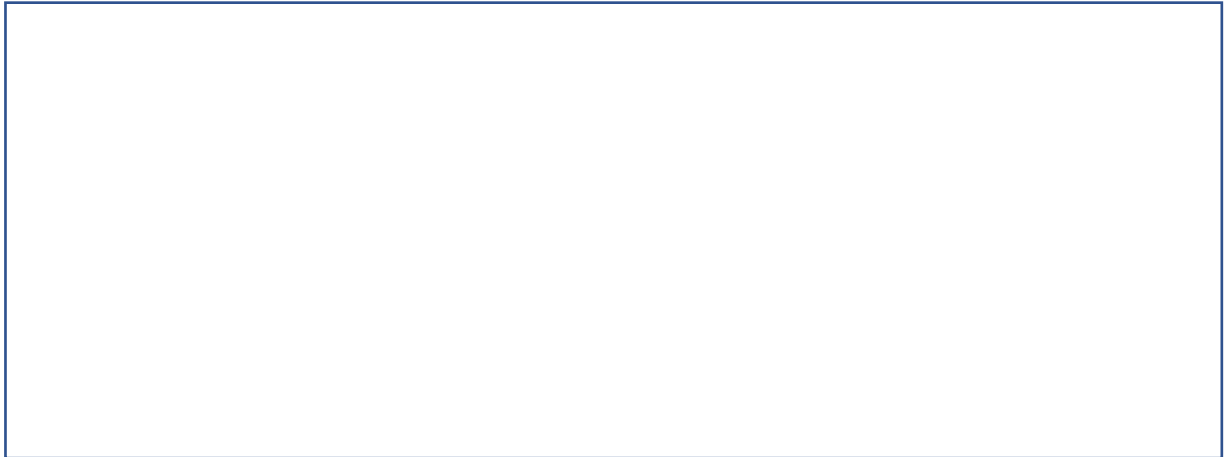


Figure 4(d)

Final state:

4. (continue...)

- (c) Assuming a Blocks World with 3 blocks A, B, C. Give an example where transiting from the initial state to the final state cannot be done in fewer than 4 moves. Draw the initial state and the final state. [5 marks]



- (d) We now augment the Blocks World by numbering the towers 0, 1, 2, ..., and introducing a new action called **puton(t1,t2)** which works like the move action, but with parameters t1 and t2 referring to the tower numbers. The action **puton(t1,t2)** moves the top block of tower t1 onto the top of tower t2, creating tower t2 if it does not exist.

For example, to transit from the initial state in Figure 4(e) to the final state in Figure 4(f), the following sequence of actions is needed: puton(0,1); puton(0,2); puton(0,2); puton(1,0); puton(2,0); puton(2,1).

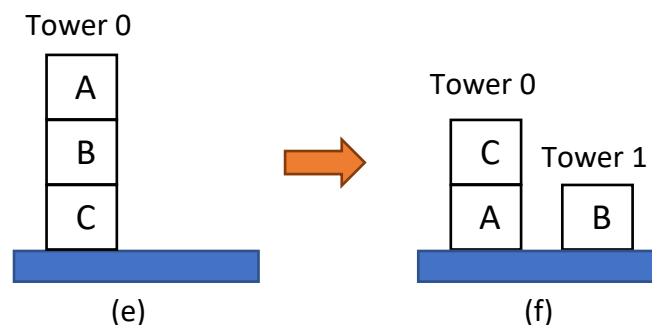


Figure 4

Write an algorithm **rotate(k)** to rotate the blocks in tower 0 by k blocks, that is, move the top k blocks to the bottom. For example, if the initial state is shown in Figure 4(g) and $k=2$, then rotating the blocks in tower 0 by 2 blocks results in the state shown in Figure 4(h). Your algorithm must work for a tower with any number of blocks.

You can access the size of a tower (that is, the number of blocks in that tower) by using the function **size(t)** where t is the tower number. You may assume that k is not greater than the size of tower 0. [10 marks]

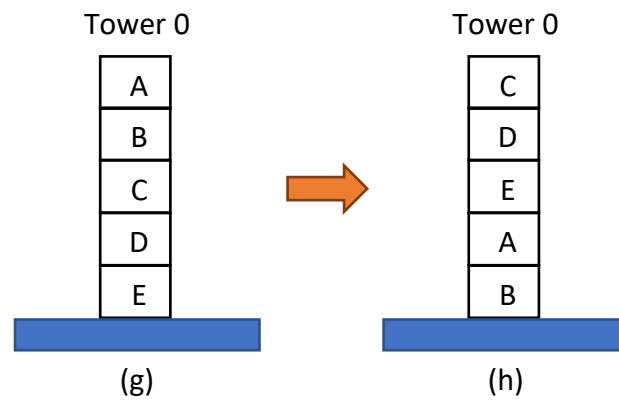


Figure 4

Section B

INSTRUCTIONS

- For SM2 students, you are to answer the questions in this section only in C programs.
- For CS students taking CS1101S, you may choose to answer the questions in this section in C/C++, Python or Java.

5. [7 marks]

If a k -digit non-negative integer is equal to the sum of the k^{th} power of its digit, it is called an *Armstrong number*. For example: 1, 153 and 1634 are Armstrong number, as $1^1 = 1$, $1^3 + 5^3 + 3^3 = 153$, and $1^4 + 6^4 + 3^4 + 4^4 = 1634$.

Write a function **is_Armstrong(n)**, where n is a non-negative integer, to return true if n is an Armstrong number, or false otherwise. You may use math library function.

(For CS1101S students) Your choice of programming language: _____

6. [8 marks]

A palindromic number is a number that reads the same from both ends. For example, 0, 9, 55, 828, 123321, and 1234321 are palindromic numbers. Leading zeroes are not allowed, for example, 0123210, which has to be entered as 123210, which is not a palindromic number.

Write a function **is_palindromic(n)** that checks if the non-negative integer n is palindromic or not. You have to use recursion in your code. You are not allowed to use data structure such as string or array. You may write other supporting function(s).

(For CS1101S students) Your choice of programming language: _____

7. [10 marks]

Given an unsorted n -element ($n > 0$) integer array (for C/Java) or list (for Python), you are to find the k th smallest element in this array/list, where $1 \leq k \leq n$. For example, if the array/list contains 17, 20, 5, 12, 2, 7, 9, 15 and $k=3$, then the 3rd smallest element is 7.

A **partition(arr, left, right)** function has been given which takes a part of an array/list arr from the left index through the right index inclusive, and partitions it as follows: (1) an element is chosen as the pivot, and (2) all elements in the array/list are rearranged such that elements smaller than the pivot are moved to the left of the pivot and elements larger than or equal to the pivot are moved to the right of the pivot. This partition function returns the new location of the pivot in the array/list. You do not need to write this function.

Using this **partition** function, write a function **find_kth** to return the k th smallest element of an unsorted array/list, without using any sorting method.

(For CS1101S students) Your choice of programming language: _____

8. [15 marks]

- (a) Write a function **reverse(str, left, right)** to reverse the characters in a string from index left through right.

For example, if str is "programming", then reverse(str, 0, 10) → "gnimmargorp".

[5 marks]

(For CS1101S students) Your choice of programming language: _____

8. (continue...)
- (b) Given a string consisting of only letters and spaces, write a function **reverse_string** to reverse the string word by word. A word is defined as a sequence of non-space characters. You may assume that the string does not begin or end with a space, and two words are separated by a space. For example,
- “The fox jumps over the lazy dog” → “dog lazy the over jumps fox The”;
“programming” → “programming”.
- You are to use the **reverse** function you have written in part (a). You do not need to write out the **reverse** function in part (a) again. [10 marks]

9. [20 marks]

The goal of a Sudoku puzzle is to fill a 9×9 grid with numbers so that each row, column and 3×3 section contain all of the digits between 1 and 9.

Figure 9(a) shows a Sudoku puzzle and figure 9(b) shows the solution.

					1	2	3	
1	2	3			8		4	
8		4			7	6	5	
7	6	5						
						1	2	3
	1	2	3			8		4
	8		4			7	6	5
	7	6	5					

Figure 9(a)

6	5	7	9	4	1	2	3	8
1	2	3	6	5	8	9	4	7
8	9	4	2	3	7	6	5	1
7	6	5	1	2	3	4	8	9
2	3	1	8	9	4	5	7	6
9	4	8	7	6	5	1	2	3
5	1	2	3	7	6	8	9	4
3	8	9	4	1	2	7	6	5
4	7	6	5	8	9	3	1	2

Figure 9(b)

Write a function **valid_Sudoku(arr)** that takes in a 9×9 integer array **arr** representing a Sudoku solution and checks if the solution is valid.

(For CS1101S students) Your choice of programming language: _____

(You may continue your program on the next page.)

9. (continue...)

This page is intentionally left blank.

--- END OF PAPER ---